1) Enlace del Repositorio: https://github.com/vickkk28/Agencia-Turismo-POO.git

Propuesta Seleccionada: Agencia de Turismo.

Integrantes:
- Victoria Garcia Restaino.
- Romina Analia Colqui.
- Nayla Dinara Granelli.

2) REQUISITOS CONSIDERADOS PARA EL DESARROLLO DEL SISTEMA

Para el desarrollo del sistema, se tomarán como base las reglas de negocio establecidas en la consigna de la Agencia de Turismo.

Tours y salidas:

- Cada tour tendrá un itinerario, una duración, una tarifa y una categoría.
- Los tours podrán ser aventuras, culturales o en la naturaleza.
- Cada salida tendrá una fecha, un cupo mínimo y máximo, un guía y un vehículo asignados.
- Una salida se confirmará al alcanzar el cupo mínimo. Si no lo alcanza siete días antes, se cancelará y se devolverá el 100% de lo pagado.
- Al alcanzar el cupo máximo, no se permitirán nuevas reservas.
- Los guías deberán estar habilitados para la categoría del tour y no podrán tener salidas superpuestas.
- La capacidad del vehículo deberá ser suficiente para la cantidad de pasajeros.

Reservas y precios

- Cada reserva estará asociada a un cliente titular y a una salida, y deberá incluir al menos un pasajero.
- Los adultos pagarán el 100% de la tarifa base, los menores de 12 años, el 60% y, los menores de 3 años, viajarán sin cargo. Los jubilados tendrán un descuento del 10%.
- Se aplicarán las promociones por reserva anticipada o por grupo, sin acumular descuentos.
- Los servicios adicionales se sumarán al precio por pasajero.
- El seguro de viaje será obligatorio para los tours de aventura.
- La reserva se confirmará con el pago de una seña del 30%.
- El saldo deberá pagarse hasta un mes antes de la salida. Si no se abona, la reserva se anulará y los lugares volverán a estar disponibles.
- Los pagos con tarjeta en cuotas tendrán un recargo del 7%.

Cancelaciones y modificaciones

- Las devoluciones dependen de la anticipación con la que se cancele la reserva: 90% hasta 30 días antes, 50% entre 29 y 15 días, 20 % entre 14 y 7 días y sin devolución con menos de 7 días.
- Si la cancelación es responsabilidad de la agencia, se devolverá el 100% de lo pagado.

- Las reservas pagadas no se eliminarán. Las correcciones se registrarán mediante notas de ajuste.

Operación y liquidaciones

- Antes de cada salida se deberá generar una lista de pasajeros con sus documentos, seguro médico y contactos de emergencia.
- No se podrá iniciar una salida sin un guía asignado ni sin la lista de pasajeros.
- Los guías cobrarán un monto fijo por cada día de una salida finalizada, más el 5% de las ventas de servicios adicionales correspondientes a esa salida.

Estas reglas se utilizarán como referencia para definir el funcionamiento del sistema y validar que las operaciones respeten las condiciones establecidas.

3) DESCRIPCIÓN DE LOS CASOS DE USO

Para el sistema de la agencia de turismo, se identifican los siguientes actores y funcionalidades.

- *Administrador:* Administrar tours, programar salidas, registrar clientes, asignar guías y vehículos, consultar reportes.
- *Vendedor:* Registrar clientes, crear y modificar reservas, calcular precios, registrar pagos y gestionar cancelaciones.
- *Coordinador:* Consultar salidas, emitir listas de pasajeros y verificar que cada salida tenga guía asignado.
- *Guia Turístico:* Consultar las salidas asignadas.
- *Responsable de liquidaciones:* Calcular honorarios de guías y consultar ingresos y ventas.

3) CASOS DE USO MÁS IMPORTANTES

CU01: Administrar tours

- Actor principal: Administrador (Permite registrar, consultar y modificar los tours, incluyendo su categoría, itinerario, duración y tarifas).

CU02: Programar salidas

- Actor principal: Administrador (Permite crear una salida para un tour, indicando fecha, cupos, guía y vehículo).

CU03: Registrar clientes

- Actor principal: Vendedor (Permite registrar los datos personales y de contacto de los clientes).

CU04: Gestionar reservas

- Actor principal: Vendedor (Permite crear reservas para una salida, registrar a los pasajeros y controlar la disponibilidad de cupos).

CU05: Calcular precio de reserva

- Actor principal: Vendedor (Permite calcular el precio total considerando la tarifa del tour, la temporada, el tipo de pasajero, los descuentos y los servicios adicionales).

CU06: Registrar pagos

- Actor principal: Vendedor (Permite registrar la seña, los pagos del saldo y las cuotas, teniendo en cuenta el medio de pago y los recargos correspondientes).

CU07: Cancelar o modificar reservas

- Actor principal: Vendedor (Permite cancelar o modificar una reserva, aplicando las políticas de devolución y las condiciones establecidas).

CU08: Organizar salidas

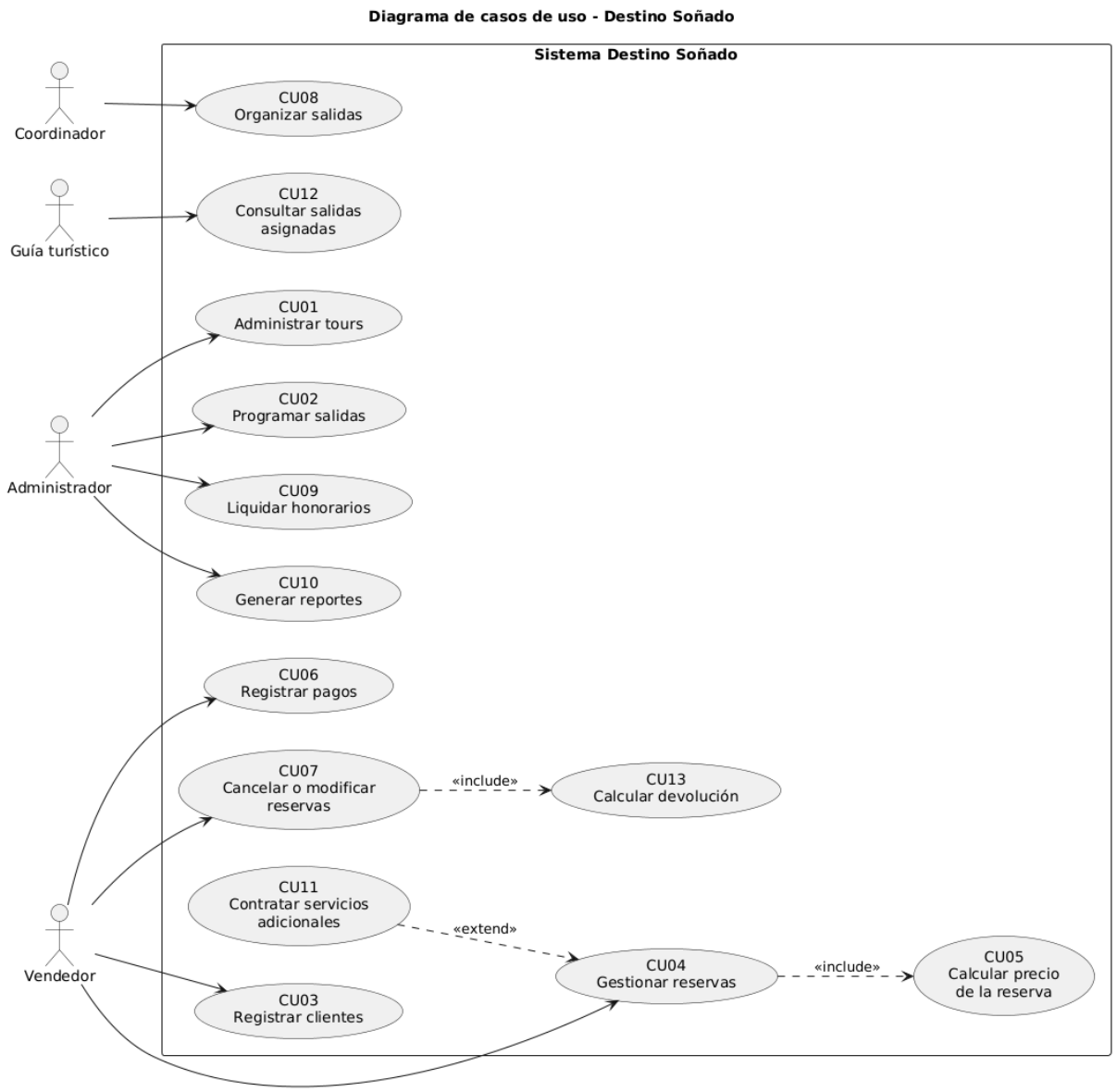
- Actor principal: Coordinador (Permite verificar las asignaciones de guías y vehículos y generar la lista de pasajeros antes de cada salida).

CU09: Liquidar honorarios

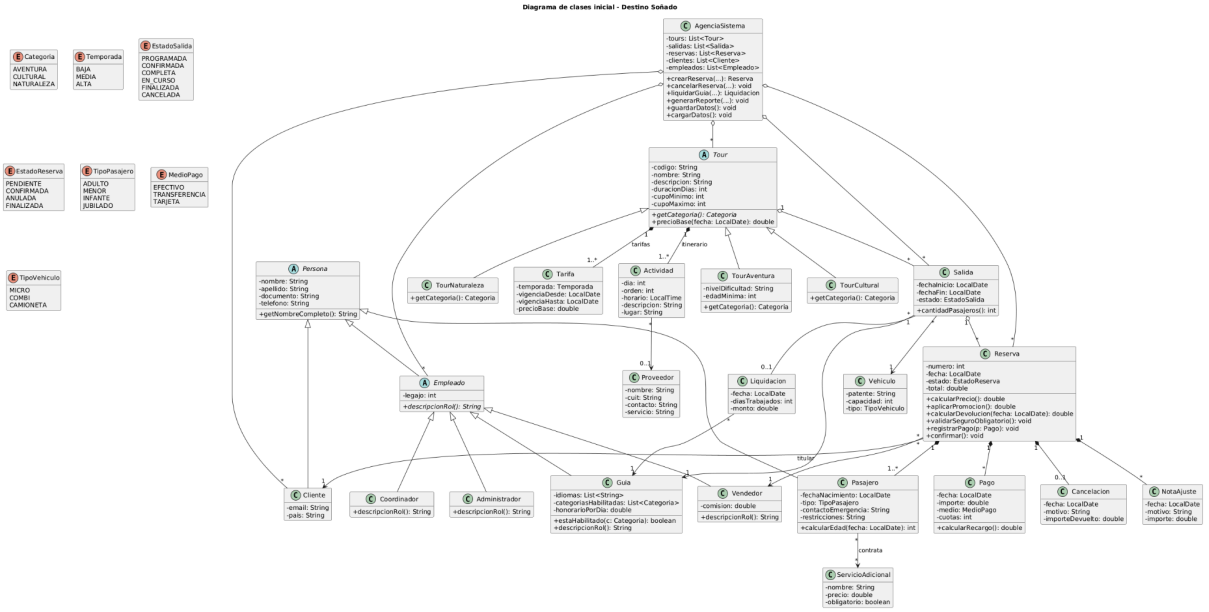
- Actor principal: Responsable de liquidaciones (Permite calcular los honorarios de los guías una vez finalizada la salida, considerando el monto fijo por día y el porcentaje de las ventas de servicios adicionales).

CU10: Generar reportes

- Actor principal: Administrador (Permite consultar reportes de ocupación por salida, ventas por vendedor, ingresos por tour y cancelaciones).

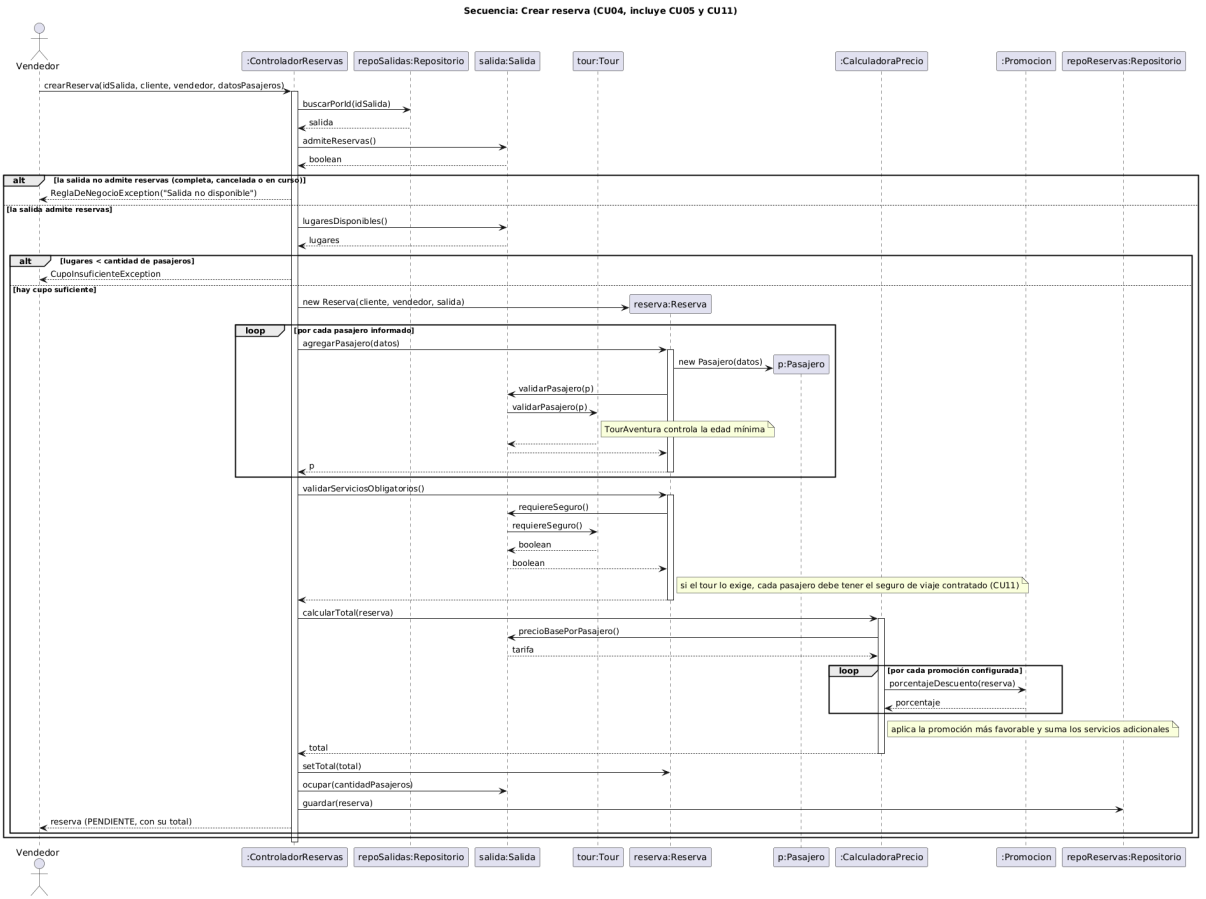

Diagrama de casos de uso - Destino Soñado
Sistema Destino Soñado
Coordinador
Guía turístico
Administrador
Vendedor
CU08
Organizar salidas
CU12
Consultar salidas
asignadas
CU01
Administrar tours
CU02
Programar salidas
CU09
Liquidar honorarios
CU10
Generar reportes
CU06
Registrar pagos
CU07
Cancelar o modificar
reservas
«include»
CU13
Calcular devolución
CU11
Contratar servicios
adicionales
«extend»
CU04
Gestionar reservas
«include»
CU05
Calcular precio
de la reserva
CU03
Registrar clientes

4)

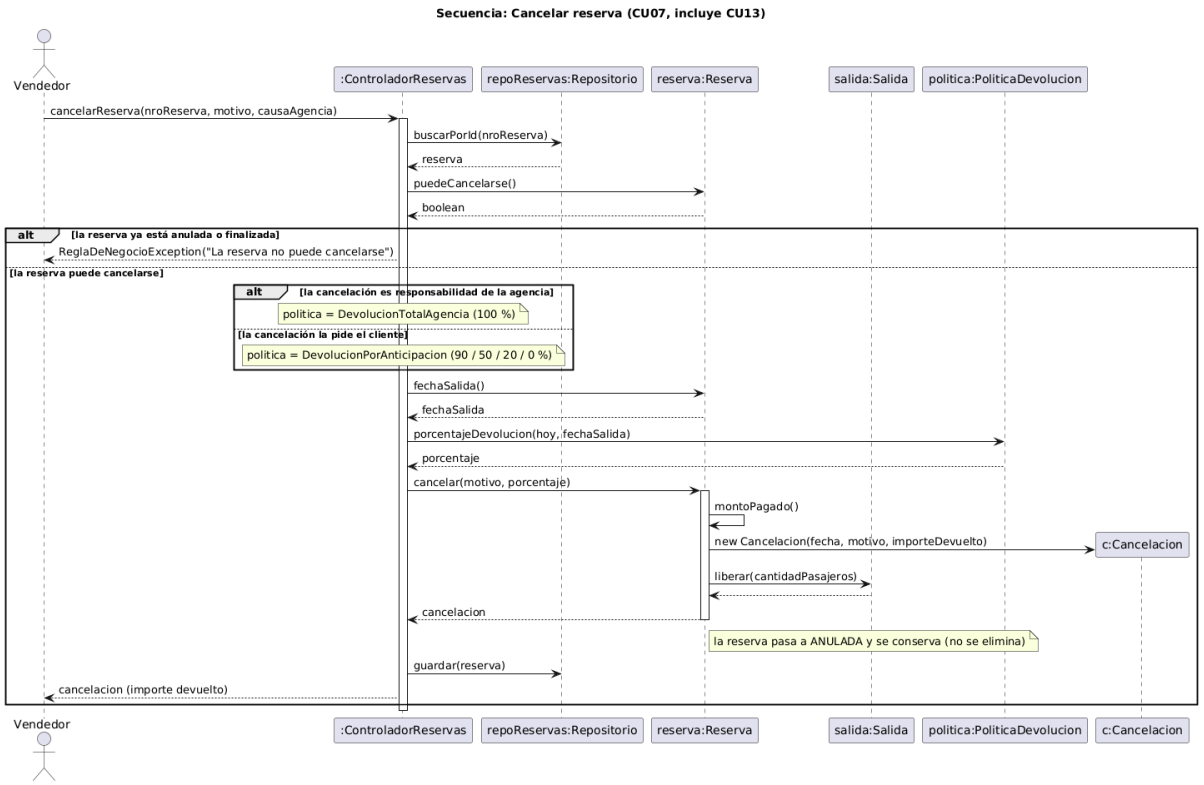


5)

SECUENCIA: Crear reserva.



SECUENCIA: Cancelar reserva.



6) Principios SOLID:
   - SRP (responsabilidad única):
     - Problema: “Reserva” calculaba precios y devoluciones, y “AgenciaSistema” hacía de todo.
     - Decision: Se extrajeron “CalculadoraPrecio”, “PoliticaDevolucion”, “LiquidadorHonorarios” y “ServicioReportes”, y la lógica de la agencia pasó a controladores.
     - Clases: “Reserva”, “CalculadoraPrecio”, “LiquidadorHonorarios”, “ServicioReportes”, “ControladorReservas”, “ControladorSalidas”.
     - Ventaja: Cada clase tiene un solo motivo para cambiar. Un cambio en las promociones no toca a “Reserva”.

   - OCP (abierto/cerrado):
     - Problema: Agregar una promoción o una política de devolución nueva obligaba a modificar los “if” de “Reserva”.
     - Decisión: Se crearon las interfaces “Promocion” y “PoliticaDevolucion”, con una clase por cada variante.
     - Clases: “Promocion”; “PromocionReservaAnticipada”, “PromocionGrupo”, “PoliticaDevolucion”, “DevolucionPorAnticipacion”, “DevolucionTotalAgencia”, “CalculadoraPrecio”.
     - Ventaja: Se agrega una promoción nueva con una clase nueva, sin modificar “CalculadoraPrecio”.

   - DIP (inversión de dependencias):
     - Problema: La persistencia en archivos estaba escrita dentro de “AgenciaSistema”, y la lógica depende de ese detalle.

- Decisión: Se creó la interfaz “Repositorio<T>” y su implementación “RepositorioArchivo<T>”. Los controladores dependen solo de la interfaz.
- Clases: “Repositorio”, “RepositorioArchivo”, “ControladorReservas”, “ControladorSalidas”, “ServicioReportes”.
- Ventajas: Se puede cambiar el almacenamiento o usar repositorios falsos en las pruebas unitarias sin tocar la lógica.

Patrones GRASP

- Experto en información:
  - Problema: El cupo y el estado de la salida se calculaban fuera de “Salida”.
  - Decisión: Cada clase resuelve lo que conoce: “Salida” sabe “lugaresDisponibles()”, “admiteReservas()” y “ventasAdicionales()”. “Reserva” sabe “montoPagado()” y “saldo()”.
  - Ventaja: La información y la lógica que la usa quedan juntas, con menos acoplamiento.

- Creador:
  - Problema: “AgenciaSistema” creaba todos los objetos.
  - Decisión: “Tour” crea sus “Salida” (ProgramarSalida), “Reserva” crea sus “Pasajero” (agregaPasajero) y su “Cancelacion” (cancelar), porque cada una los contiene o tiene los datos para inicializarlos.
  - Ventajas: La creación queda donde están los datos y se evita una clase que lo conoce todo.

- Controlador:
  - Problema: La interfaz con el usuario hablaba directamente con las clases del dominio a través de “AgenciaSistema”.
  - Decisión: “ControladorReservas” y “ControladorSalidas” reciben las operaciones de la interfaz y las coordinan.
  - Ventajas: El dominio no depende de la interfaz, cambia de consola a interfaz gráfica (no toca el modelo).

- Polimorfismo:
  - Problema: “Reserva” usaba “instanceof” para saber si el tour era de aventura, y “Pago” usaba un “if” según el medio.
  - Decision: “Tour” define “requiereSeguro()” y “validarPasajero()”, que “TourAventura” sobrescribe. “Pago” define “calcularRecargo()” como método abstracto, que sobrescriben “PagoEfectivo”, “PagoTransferencia” y “PagoTarjeta”.
  - Ventaja: Desaparecen los condicionales por tipo y agregar un nuevo tipo de tour o de pago no obliga a modificar el código existente.

7)

- Se eliminó “AgenciaSistema”: Sus responsabilidades pasaron a “ControladorReservas”, “ControladorSalidas”, “LiquidadorHonorarios”, “ServicioReportes” y los repositorios (SRP y Controlador).
- Se agregaron “CalculadoraPrecio” y la interfaz “Promocion” con “PromocionReservaAnticipada” y “PromocionGrupo”. “Reserva” ya no calcula precios (SRP y OCP).
- Se agregaron “PoliticaDevolucion”, “DevolucionPorAnticipacion” y “DevolucionTotalAgencia”. “Reserva” ya no calcula devoluciones, solo recibe el porcentaje (OCP).
- “Pago” pasó a ser abstracta con “PagoEfectivo”, “PagoTransferencia” y “PagoTarjeta”, cada una con su “calcularRecargo()” (Polimorfismo). El enum “MedioPago” ya no es necesario.
- “Tour” incorpora “requiereSeguro()”, “validarPasajero()” y “programarSalida()”. “TourAventura” sobrescribe los dos primeros (Polimorfismo y Creador).
- “Salida” incorpora “admiteReservas()”, “lugaresDisponibles()”, “ocupar()”, “liberar()”, “ventasAdicionales()” y “confirmarSiCorresponde()” (Experto).
- “Reserva” incorpora “agregarPasajero()”, “montoPagado()”, “saldo()”, “puedeCancelarse()” y “cancelar()”. Crea sus “Pasajero” y su “Cancelacion” (Experto y Creador).
- Se agregó “DatosPasajero” como clase simple para llevar los datos desde la interfaz hasta “Reserva.agregarPasajero()”.
- Se agregaron “Repositorio<T>” y “RepositorioArchivo<T>” para la persistencia (DIP).
- Se agregaron “ReglaDeNegocioException” y “CupoInsuficienteException” para informar las violaciones de reglas de negocio.
- “TipoPasajero” incorpora “factorTarifa()” (adulto 1,0; menor 0,6; infante 0; jubilado 0,9) y reemplaza los condicionales por tipo.

